

Módulo V: Taller Integrativo

Clase 4 — Dashboard completo con vibe coding

2026-07-01

Contacto

-  sebaegana@gmail.com
 -  <https://segana.netlify.app>
 -  <https://www.linkedin.com/in/sebastian-egana-santibanez/>
 -  <https://github.com/sebaegana>
-

Módulo V

Clase 4: Dashboard completo con vibe coding

Sidebar · Plotly · Semáforo · Vibe coding en práctica

Objetivo de la sesión

- Agregar sidebar con filtros interactivos a la app
 - Incorporar gráficos de tendencia y cumplimiento con Plotly
 - Implementar semáforo de cumplimiento de metas
 - Practicar vibe coding: cada grupo amplía su app con el LLM
-

Dónde quedamos

Al final de la Clase 3 cada grupo tiene:

```
app.py
    st.title()          ← título de la app
    pd.read_csv()       ← carga de datos KPIs
    st.metric()         ← tarjetas por indicador
```

Hoy construimos sobre eso.

Agregar sidebar con filtros

```
with st.sidebar:
    st.markdown("## Filtros")

    # Filtro por perspectiva BSC
    perspectivas = ["Todas"] + sorted(df["perspectiva"].unique().tolist())
    perspectiva_sel = st.selectbox("Perspectiva", perspectivas)

    # Filtro por período
    periodos = sorted(df["periodo"].unique().tolist())
    periodo_sel = st.selectbox("Período", periodos,
                              index=len(periodos) - 1)

# Aplicar filtros al DataFrame
df_filtrado = df[df["periodo"] == periodo_sel]
if perspectiva_sel != "Todas":
    df_filtrado = df_filtrado[
        df_filtrado["perspectiva"] == perspectiva_sel
    ]
```

Gráfico de tendencia con Plotly

```
import plotly.graph_objects as go

# El usuario elige el indicador a visualizar
indicador_sel = st.selectbox("Indicador", df["indicador"].unique())

df_ind = df[df["indicador"] == indicador_sel].sort_values("mes")
meta    = df_ind["meta"].iloc[0]
unidad  = df_ind["unidad"].iloc[0]
```

```

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=df_ind["periodo"], y=df_ind["valor"],
    mode="lines+markers",
    line=dict(color="#1D5C35", width=3),
))
fig.add_hline(y=meta, line_dash="dash", line_color="red",
              annotation_text=f"Meta: {meta} {unidad}")

st.plotly_chart(fig, use_container_width=True)

```

Gráfico de cumplimiento con Plotly Express

```

import plotly.express as px

df_ult = df[df["periodo"] == periodo_sel].copy()

#% de cumplimiento: distinto según si mayor es mejor o no
df_ult["cumplimiento"] = df_ult.apply(
    lambda r: min(r["valor"] / r["meta"] * 100, 150)
    if r["mayor_es_mejor"]
    else min(r["meta"] / r["valor"] * 100, 150),
    axis=1,
)

fig_bar = px.bar(
    df_ult.sort_values("cumplimiento"),
    x="cumplimiento", y="indicador",
    orientation="h", color="perspectiva",
    title="% de cumplimiento por indicador",
)
fig_bar.add_vline(x=100, line_dash="dash", line_color="red")

st.plotly_chart(fig_bar, use_container_width=True)

```

Semáforo de cumplimiento

```

def semaforo(valor, meta, mayor_es_mejor):
    ratio = valor / meta if meta != 0 else 0
    if mayor_es_mejor:
        if ratio >= 1.0: return " "
        elif ratio >= 0.85: return " "

```

```

        else:
            return " "
    else:
        if ratio <= 1.0:
            return " "
        elif ratio <= 1.15:
            return " "
        else:
            return " "

# Aplicar a las métricas del período
cols = st.columns(len(df_filtrado))
for col, (_, row) in zip(cols, df_filtrado.iterrows()):
    icono = semaforo(row["valor"], row["meta"], row["mayor_es_mejor"])
    delta = row["valor"] - row["meta"]
    col.metric(
        label=f"{icono} {row['indicador']}",
        value=f"{row['valor']} {row['unidad']}",
        delta=f"{delta:+.1f} vs meta",
        delta_color="normal" if row["mayor_es_mejor"] else "inverse",
    )

```

Tabla con semáforo

```

st.subheader("Tabla de cumplimiento")

filas = []
for _, row in df_filtrado.iterrows():
    icono = semaforo(row["valor"], row["meta"], row["mayor_es_mejor"])
    filas.append({
        "Estado": icono,
        "Indicador": row["indicador"],
        "Perspectiva": row["perspectiva"],
        "Valor actual": f"{row['valor']} {row['unidad']}",
        "Meta": f"{row['meta']} {row['unidad']}",
        "Desviación": f"{row['valor'] - row['meta']:+.1f}",
    })

st.dataframe(pd.DataFrame(filas),
             use_container_width=True,
             hide_index=True)

```

Vibe coding en práctica

Cada grupo le pide al LLM (Continue.dev) una funcionalidad nueva para su app.

Ejemplos de prompts para practicar:

“Agrega una sección que muestre el indicador con peor cumplimiento del período seleccionado, con un texto de alerta en rojo.”

“Agrega un `st.download_button` que permita descargar la tabla de cumplimiento como CSV.”

“Crea una segunda página en la app usando `st.tabs` con una pestaña para el dashboard y otra para el detalle de un indicador.”

Estructura de la app al final de esta clase

`app.py`

```
Sidebar: filtros perspectiva + período  
st.metric() con semáforo por indicador  
Gráfico de tendencia (Plotly)  
Gráfico de cumplimiento (Plotly Express)  
Tabla semáforo (st.dataframe)
```

Todo con los datos reales del proyecto integrador.

Trabajo de la sesión

1. Agregar sidebar con filtros a la app de la Clase 3
 2. Implementar la función `semaforo()` en las métricas
 3. Agregar el gráfico de tendencia
 4. Agregar el gráfico de barras de cumplimiento
 5. Usar `Continue.dev` para agregar una funcionalidad nueva
-

Próxima clase

Clase 5 — Deploy en Streamlit Cloud vía GitHub

- Crear repositorio en GitHub desde VS Code
- Subir el código del proyecto
- Conectar con Streamlit Community Cloud
- Obtener la URL pública del dashboard